Trabajo Práctico No. 2

Opción 1: Enriquecer el caso del Trabajo Práctico No. 1 con Estructuras de Árboles AVL.

Recomendaciones para la entrega del trabajo:

- Realizar un informe explicando con el siguiente contenido ...
    - Explicación del problema o situación que va a resolver (uno o dos párrafos)
    - Diseño de la solución en lenguaje natural, indicando las estructuras de datos que se utilizan para la solución, clases y los algoritmos asociados
    - Incluir un análisis de los resultados obtenidos, conclusiones y recomendaciones
- El código en lenguaje Python debe entregarse en uno o varios archivos separados .py
- El ejercicio debe involucrar el uso de árboles AVL utilizando las clases de objetos vistas en la materia o versiones mejoradas.
- Puede utilizar herramientas de IA indicando para qué las ha utilizado (por ejemplo: para qué clases/ métodos/ secciones del código, generación del informe, datos, u otros).

Opción 2: Generar otro caso diferente al del Trabajo Práctico No.1 aplicando ahora Estructuras de Árboles AVL.

Recomendaciones para la entrega del trabajo: idem opción 1.

En este caso no necesita utilizar estructuras de colas que se usaron en el TP No.1.

Opción 3: Análisis de Series Temporales (Criptoactivos)

Este enunciado se enfoca en la diferencia de rendimiento entre árboles cuando los datos ingresan con un orden relativo (lo cual es común en series de tiempo).

Enunciado del Trabajo Práctico

Título: Optimización de Consultas en Mercados Financieros mediante Árboles Balanceados.

Objetivo: Comparar el tiempo de búsqueda y la altura de las estructuras al indexar precios históricos de activos digitales.

Consigna:

1. **Fuente de Datos:** Los alumnos deberán descargar el historial de precios diarios de Bitcoin (BTC) o Ethereum (ETH) de los últimos 5 años desde **Yahoo Finance (biblioteca `yfinance` en Python)**.
2. **Implementación:**
   - Cargar los datos en un **Binary Search Tree (BST)** simple.
   - Cargar los mismos datos en un **AVL Tree**.
3. **Desafío:** Insertar los datos en dos órdenes distintos:
   - Orden cronológico (esto provocará la degeneración del BST).
   - Orden aleatorio (shuffle).
4. **Resultados esperados:** Un informe técnico que compare mediante gráficos la **altura del árbol** resultante y el **tiempo de ejecución** promedio para buscar el precio de una fecha específica en ambas estructuras.

Opción 4: Sistema de Búsqueda de Catálogo (Libros de Open Library)

Ideal para trabajar con cadenas de texto y grandes volúmenes de metadatos.

Enunciado del Trabajo Práctico

Título: Indexación y Recuperación Eficiente de Títulos Bibliográficos.

Objetivo: Desarrollar un motor de búsqueda por "Título" que minimice la profundidad de búsqueda en un catálogo extenso.

Consigna:

1. **Fuente de Datos:** Utilizar el **Open Library Data Dump** (disponible en openlibrary.org/developers/dumps). Los alumnos deben extraer una muestra de al menos 10.000 títulos de libros.
2. **Implementación:**
   - Diseñar una clase `Libro` que almacene el título y su identificador (OLID).
   - Implementar la lógica de comparación de strings para organizar los nodos alfabéticamente.
   - Programar las rotaciones necesarias para mantener el equilibrio en el **árbol AVL**.
3. **Análisis:**
   - Realizar una simulación de 1.000 búsquedas de títulos inexistentes.
   - Calcular el "Peor Caso" de búsqueda (número máximo de comparaciones) en el **BST** vs. el **AVL**.

4. **Requisito de Visualización:** El programa debe ser capaz de imprimir por consola los niveles del árbol para una sub-muestra de 15 elementos, demostrando visualmente el balanceo.